Design Document

AI-Powered Energy Consumption Optimization Assistant

1. Problem Statement

Energy analysts need to explore historical energy-consumption records, identify high-consumption or peak-period cases, retrieve similar scenarios and energy-efficiency guidance, and receive actionable recommendations. This project combines processed household energy records, a domain knowledge base, hybrid retrieval, LLM-assisted analysis, rule-based paths, and estimated cost/carbon impact calculations. The source dataset is household-focused and does not independently validate results for every commercial or industrial setting.

2. Architecture Design

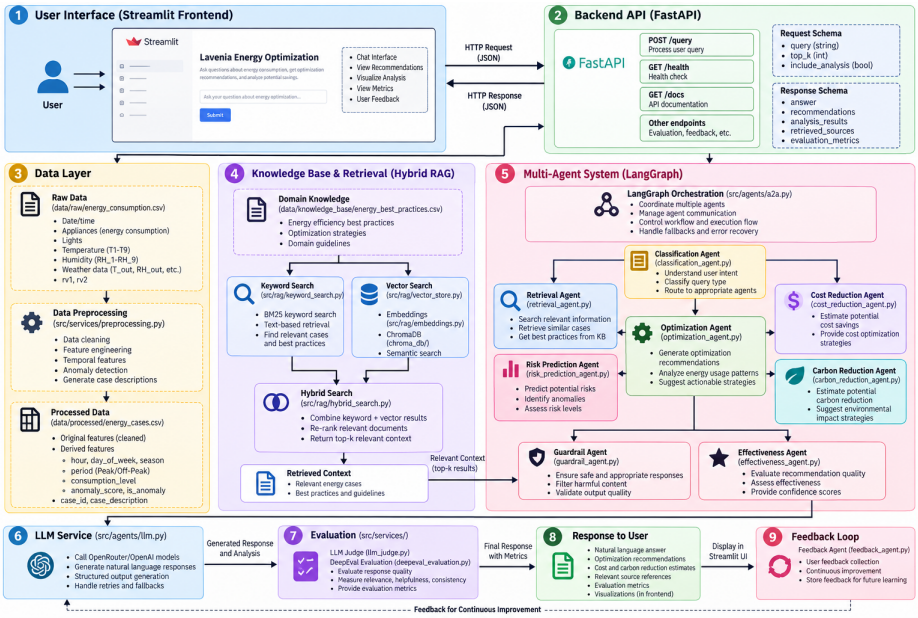


Figure 1. High-level architecture diagram supplied for the project.

High-level flow: Dataset → Preprocessing → Record-level case creation → Embeddings → ChromaDB and keyword retrieval → Hybrid search → Retrieved context → FastAPI/coordinator and specialist agents → LLM-assisted analysis with rule-based paths → Evaluation components → Streamlit response and feedback.

The Streamlit interface sends user queries to FastAPI. Preprocessing converts raw data into processed energy cases. The retrieval layer searches energy cases and knowledge-base entries. The coordinator in src/agents/a2a.py manages the specialist-agent workflow. Results are returned to the interface, and feedback routes support feedback collection and summaries.

3. Technology Selection

LLM — OpenRouter with the configured model openai/gpt-4.1: Supports natural-language analysis and recommendations, and judge-based evaluation when enabled. External calls are configuration-controlled; rule-based paths support selected workflows.

Embedding model — all-MiniLM-L6-v2: Converts searchable case descriptions and knowledge entries into vectors for semantic similarity search.

Vector database — ChromaDB: Stores embeddings and supports similarity-based retrieval.

Frameworks and libraries: Python and Pandas support preprocessing and agent logic; FastAPI and Pydantic provide API routes and request validation; Streamlit provides the user interface; LangGraph/workflow code supports orchestration. Do not assume every specialist agent is a separate LangGraph node unless the graph source explicitly defines it.

Evaluation tools — LLM-as-a-judge and DeepEval: Assess generated answer and recommendation quality using configured evaluation dimensions and metrics.

4. Retrieval Strategy

Chunking strategy: Record-level/case-level segmentation is used rather than fixed-token or overlapping-window splitting. Each processed energy case is represented as one searchable text description with its fields/metadata; each knowledge-base CSV entry is a separate searchable item. The recorded preprocessing/ingestion output reported 19,735 processed energy cases. This is the energy-case count for that run, not necessarily the combined count of cases plus knowledge-base entries.

Embedding approach: all-MiniLM-L6-v2 creates vector representations for searchable case descriptions and knowledge entries, which are stored/queryable through ChromaDB.

Retrieval method: Semantic vector search is provided through the vector-store layer. Keyword retrieval is implemented in src/rag/keyword_search.py. src/rag/hybrid_search.py combines/ranks results and returns top-k context for downstream analysis.

Hybrid search: The documented project weighting is approximately 70% semantic relevance and 30% keyword relevance. This is a project-specific configuration, not a universal optimum.

5. Agent Design

The project has nine named specialist agents. The coordinator/orchestration component in src/agents/a2a.py coordinates the workflow and is not counted as a tenth specialist agent.

- **Guardrail Agent:** Checks query suitability and supports appropriate responses.
- **Classification Agent:** Classifies query intent/type to guide the analysis path.
- **Retrieval Agent:** Retrieves relevant energy cases and knowledge-base context.
- **Risk Prediction Agent:** Identifies potential risk/anomaly signals from available features and configured logic.
- **Optimization Agent:** Develops actionable energy optimization recommendations.
- **Cost Reduction Agent:** Estimates potential cost impact using consumption and tariff assumptions.
- **Carbon Reduction Agent:** Estimates potential emissions impact using an emission-factor assumption.
- **Effectiveness Agent:** Assesses recommendation quality/effectiveness using configured logic.
- **Feedback Agent:** Collects and summarizes user feedback for review.

Workflow and communication: A query enters through the UI/API, passes input checks and classification, and triggers retrieval. The coordinator invokes relevant analysis/recommendation components; risk, cost, carbon and effectiveness logic can support the recommendation. LLM generation is used when enabled, with rule-based paths in supported workflows. Results return to Streamlit, and feedback is submitted through feedback routes. The high-level diagram does not mean every agent executes for every query.

6. Evaluation Strategy

LLM-as-a-judge: Assesses relevance, actionability, evidence support, energy-saving suitability and overall quality. A previously recorded run returned overall 0.70, with relevance 0.90, actionability 0.80, evidence support 0.50 and suitability 0.90. These are results from that specific run, not guaranteed scores for every query.

DeepEval: A previously recorded run returned overall 0.825, Answer Relevancy 0.75 and Energy Recommendation Quality 0.90. These are run-specific outputs and should be reported with their configuration.

Test cases and metrics: Project tests cover API behavior, preprocessing, guardrails, hybrid/keyword retrieval, agents, feedback, LLM recommendations and evaluation components. Run the current suite before submission and report actual pass/fail counts. Recorded quality metrics include Answer Relevancy, Energy Recommendation Quality and judge dimensions such as relevance, actionability, evidence support, suitability and overall quality. Retrieval metrics based on rule-derived labels are proxy measurements, not independently human-verified ground truth. Cost/carbon outputs are estimates, not measured savings.

7. Challenges and Trade-offs

- **Hybrid retrieval:** Semantic search captures conceptual similarity; keyword matching helps with exact terms. The 70/30 balance may need tuning against representative queries.
- **Case-level segmentation:** Preserves each record and its metadata as a searchable unit, but does not merge adjacent timestamps into longer temporal narratives.
- **Dataset scope:** Household energy data cannot independently validate recommendations for all commercial or industrial settings.
- **Impact estimates:** Cost/carbon results depend on supplied baseline, tariff, emission factor and reduction-rate assumptions.
- **LLM reliability and cost:** Provider availability, rate limits and credentials can affect generation and judge runs; configurable evaluation and rule-based paths help manage cost and resilience.
- **Evaluation limitations:** Judge scores depend on the judge model, and rule-derived retrieval labels are imperfect proxies.